

UD 4. Manejo de datos externos con PHP

Índice de contenidos

0. Introducción.....	2
1. Cookies.....	2
1.1. Etapas de uso de las cookies.....	2
1.2. Envíos de cookies en cabeceras.....	2
1.3. Función setcookie().....	3
1.4. Array \$_COOKIE[].....	3
1.5. Disponibilidad de las cookies.....	4
2. Manejo de sesiones.....	6
2.1. Identificador de sesión (SID).....	6
2.2. Automatización del uso de sesiones en PHP.....	7
2.3. Configuración de sesiones.....	7
2.4. Inicio y fin de una sesión.....	8
2.5. Identificadores de sesión personalizados.....	9
3. Autenticación de usuarios y control de acceso.....	11
3.1. Mecanismos de autenticación.....	12
3.1.1. Protocolo HTTP.....	12
3.1.2. Formulario web.....	13
3.2 Incorporación de métodos de autenticación a una aplicación web.....	14

0. Introducción

Hasta ahora hemos manejado varios tipos de datos ubicados en **almacenes** distintos como variables de usuario, variables superglobales o datos obtenidos desde columnas de registros de tablas de BBDD. Existen otras opciones más en PHP para manejar datos que el usuario ofrece al servidor para una completa interacción con el mismo.

Además del lugar de almacenamiento otro factor importante es el **ámbito** de existencia de ese dato así como la **temporalidad**, medida en términos de persistencia o no en el sistema.

En esta unidad veremos cookies, variables de sesión y variables de autenticación de usuario.

1. Cookies

Una **cookie** es una variable tratada en un sitio web que se guarda en el entorno del navegador del usuario (normalmente en un fichero de texto o una pequeña base de datos). Su uso más típico es el almacenamiento de las preferencias del usuario (por ejemplo, el idioma en que se deben mostrar las páginas), para que no tenga que volver a indicarlo la próxima vez que visite el sitio.

1.1. Etapas de uso de las cookies

En el proceso de uso de la cookie se establecen **dos etapas**:

- El sitio web crea la cookie, si el usuario tiene permitido el uso de cookies.
- El sitio web lee, en las comunicaciones de cabecera de las peticiones HTTP del cliente, el valor de las cookies que para ese sitio tiene el usuario guardadas.

Para saber más ...

Las herramientas para desarrolladores de Mozilla Firefox o Google Chrome te permiten ver, borrar e incluso editar las Cookies.

1.2. Envíos de cookies en cabeceras

Para la creación de cookies el servidor, al responder a peticiones HTTP del cliente, le puede indicar en su respuesta que almacene una o varias cookies. La indicación o no de ello está en manos del desarrollador del sitio web que ofrece ese servidor:

```
HTTP/2.0 200 OK
Content-Type: text/html
Set-Cookie: yummy_cookie=choco
Set-Cookie: tasty_cookie=strawberry
```

[cuerpo]

Los servidores web le indican al cliente que almacene una cookie emitiendo un encabezado HTTP especial, "Set-Cookie".

Una vez el navegador tiene las cookies, cada solicitud posterior del cliente a ese servidor incluirá el encabezado HTTP, "Cookie" con los datos que se almacenaron.

```
GET /sample_page.html HTTP/2.0
Host: www.example.org
Cookie: yummy_cookie=choco; tasty_cookie=strawberry
```

1.3. Función `setcookie()`

En PHP, para almacenar una cookie en el navegador del usuario, puedes utilizar la función `setcookie()`. El único parámetro obligatorio que tienes que usar es el nombre de la cookie, pero admite varios parámetros más opcionales.

Por ejemplo, si quieres almacenar en una cookie la IP desde la que nos visitó el usuario por última vez, puedes hacer:

```
setcookie( "ultimaIP", $_SERVER['REMOTE_ADDR'], time()+3600 );
```

Los dos primeros parámetros son el **nombre de la cookie** y su **valor**. El tercero es la **fecha de caducidad** de la misma (en este ejemplo una hora medida en segundos desde el momento `time()` en que se ejecute). El único parámetro obligatorio es el primero. En caso de no definir el segundo parámetro se borra la cookie. Si el tercer parámetro es 0 la cookie se eliminará cuando se cierre el navegador. Un tiempo en el pasado elimina la cookie. Para incluir cualquier opción hay que incluir todas las anteriores.

Del resto de opciones nos puede interesar la relativa a la indicación de la ruta del servidor web donde tendrá existencia la ruta, fuera de la cual la cookie no existe.

```
setcookie("usuario", $_GET['usuario'], 0, "/carpetaCookie");
```

1.4. Array `$_COOKIE[]`

El proceso de recuperación de la información que almacena una cookie es muy simple. Cuando accedes a un sitio web, el navegador le envía de forma automática todo el contenido de las cookies que almacene relativas a ese sitio en concreto. Desde PHP puedes acceder a esta información por medio del array `$_COOKIE`.

```
echo $_COOKIE['ultimaIP'];
```

Para saber más ...

Aparte de las cookies definidas por el usuario existen otras cookies que las inicializa el servidor y no el cliente en su navegador. Más adelante veremos un caso relativo a la cookie de nombre 'PHPSESSID'

1.5. Disponibilidad de las cookies

Como el envío desde cliente a servidor de cookies se hace en la cabecera de la petición HTTP correspondiente, cuando se quiere crear una cookie será después de esa cabecera y por tanto no estará disponible hasta la siguiente recarga. Es decir, la cookie estará disponible mientras el navegador disponga de ella antes de lanzar la cabecera.

Siempre que utilices cookies en una aplicación web, debes tener en cuenta que en última instancia su **disponibilidad está controlada por el cliente**. Por ejemplo, algunos usuarios deshabilitan las cookies en el navegador porque piensan que la información que almacenan puede suponer un potencial problema de seguridad. O la información que almacenan puede llegar a perderse porque el usuario decide formatear el equipo o simplemente eliminarlas de su sistema.

Si una vez almacenada una cookie en el navegador quieres programar para eliminarla antes de que expire, puedes utilizar la misma función **setcookie()** pero indicando una fecha de caducidad anterior a la actual.

Actividad 4.1

Crea una página en la que almacenes en una cookie el último instante en que el usuario visitó la página. Si es su primera visita, muestra un mensaje de bienvenida. En caso contrario, muestra la fecha y hora de su anterior visita. Incluye como plazo de caducidad de la cookie un año (exprésalo en segundos).

Actividad 4.2

A partir del código siguiente, realiza las pruebas que se indican en las capturas del navegador.

```
<pre>
<?php
print_r($_COOKIE);
$valorCookie = $_GET['usuario'];
$rutaCookie = "/carpetaCookie/";
if ( ! isset($_COOKIE['usuario']) ) {
    setcookie("usuario", $valorCookie , time()+3, $rutaCookie);
    echo "<p>La cookie <b>\\"usuario\\"</b> se ha creado para la ruta ".
        "<b>$rutaCookie</b> con valor <b>$valorCookie</b>.</p>";
} else {
    echo "<p>La cookie <b>\\"usuario\\"</b> ya existe con valor ".
        "<b>$valorCookie</b>.</p>";
}
?>
</pre>
```

```
localhost/carpetaCookie/cookie.php?usuario=Juan

Array
(
)

La cookie "usuario" se ha creado para la ruta /carpetaCookie/ con valor Juan.
```

Primera vez (carpetaCookie)

```
localhost/carpetaCookie/cookie.php?usuario=Juan

Array
(
    [usuario] => Juan
)

La cookie "usuario" ya existe con valor Juan.
```

Siguientes veces a la primera (carpetaCookie)

```
localhost/otraCarpeta/cookie.php?usuario=Juan

Array
(
)

La cookie "usuario" se ha creado para la ruta /carpetaCookie/ con valor Juan.
```

Primera vez y siguientes (otraCarpeta)

Una de las principales desventajas de tenerlas alojadas en tu navegador es que almacenan cualquier tipo de información y por lo tanto, es complicado proteger la **privacidad**. Los datos obtenidos, en algunas ocasiones, pueden llegar a ser usados por personas ajenas con fines ilegales. Otro aspecto negativo es que se trata de una forma de ofrecer **publicidad** invasiva, ya que se tiene acceso a los datos personales de las personas.

Para evitar que el sitio web almacene información, es tan sencillo como eliminar las cookies en la sección «herramientas» y después hacer clic en «Borrar los datos de navegación». Sin embargo, no siempre es necesario rechazarlas ya que **tienen tanto ventajas como desventajas**. Por un lado, son de gran ayuda para mejorar la experiencia en Internet, ya que ayudan a crear un perfil de usuario personalizado. El problema llega cuando las marcas o determinados organismos abusan de las políticas de datos y obtienen información personal de los usuarios sin su consentimiento.

2. Manejo de sesiones

Para obtener datos externos a la aplicación web que estamos diseñando, además de cookie existen, entre otras opciones, las **variables de sesión**. El término **sesión** hace referencia al conjunto de información relativa a un usuario concreto que perdura durante un **espacio de tiempo**. Esta información puede ser tan simple como el nombre del propio usuario, o más compleja, como los artículos que ha depositado en la cesta de compra de una tienda online.

2.1. Identificador de sesión (SID)

Cada usuario distinto de un sitio web tiene su propia información de sesión. Para distinguir una sesión de otra se usan, como técnica genérica independiente del marco de desarrollo usado, los **identificadores de sesión** (SID). Un SID es un atributo que se asigna a cada una de las sesiones de cada uno de los visitantes de un sitio web y que lo identifica. De esta forma, si el servidor web puede conocer el SID de un usuario, puede relacionarlo con toda la información que posee sobre él y que se mantiene en la sesión del usuario. Esa información se almacena en el servidor web, generalmente en ficheros aunque también se pueden utilizar otros mecanismos de almacenamiento como bases de datos.

El identificador de sesión tiene una gran ventaja. Si el usuario abre varias ventanas del mismo sitio web, estas se asignarán a una sola sesión. Esto permite iniciar varias consultas de forma simultánea sin que se pierda información personal importante.

Para saber más ...

Si quieres abrir dos sesiones diferentes en un mismo equipo o dispositivo puedes “engañar” al sistema usando dos navegadores distintos, pues abrir varias ventanas en el mismo navegador las identificará como de la misma sesión.

Con los medios que hemos estudiado hasta ahora, para que el dato de la **SID esté disponible en PHP** durante la duración de la sesión y se mantenga “vivo” entre accesos a distintos recursos del sitio, tenemos dos maneras de traspasar el SID entre las páginas que visita el usuario:

- Utilizando cookies: guardamos el SID en una cookie, por ejemplo en `$_COOKIE['SID']` o `$_COOKIE['PHPSESSID']`, siempre que la sesión esté abierta, o
- también se puede propagar el SID en un parámetro de la URL. El SID se añade como una parte más de la URL, de la forma:

`http://www.misitioweb.com/tienda/listado.php?SID=34534fg4ffg34ty`

Ninguna de las dos maneras es perfecta. Ya sabes los problemas que tiene la utilización de cookies. Pese a ello, es el mejor método y el más utilizado. Propagar el SID como parte de la URL conlleva mayores desventajas, como la imposibilidad de mantener el SID entre distintas sesiones, o el hecho de que compartir la URL con otra persona implica compartir también el identificador de sesión.

En ambos casos, esto plantea un posible **problema de seguridad**. El SID puede ser conseguido por otra persona, y a partir del mismo obtener la información de la sesión del usuario. La manera más segura de utilizar sesiones es almacenando los SID en cookies y utilizar HTTPS para encriptar la información que se transmite entre el servidor web y el cliente.

Actividad 4.3

Crea una conjunto de páginas a modo de ejemplo de un sitio web, donde en todas controlemos que el valor de una cookie de nombre “nombreUsuario” sea “dwes” con valor de expiración cero La idea es que esa cookie sea una variable de sesión solamente con existencia en la sesión actual.

2.2. Automatización del uso de sesiones en PHP

Para contrarrestar los inconvenientes que presenta la gestión de los datos relativos a sesiones en PHP surgió. como mejora aparecida en la versión 4. el **proceso de manejo de sesiones en PHP, que está automatizado** en gran medida. Cuando un usuario visita un sitio web, no es necesario programar un procedimiento para ver si existe un SID previo y cargar los datos asociados con el mismo. Tampoco tienes que utilizar la función `setcookie()` si quieres almacenar los SID en cookies, o ir pasando el SID entre las páginas web de tu sitio si te decides por propagarlo. Todo esto PHP lo hace automáticamente.

Básicamente disponemos de funciones como `session_start()` y `session_destroy()` para iniciar y destruir sesiones y de la variable superglobal `$_SESSION` para consultar la información almacenada en la sesión. Más adelante veremos su uso.

2.3. Configuración de sesiones

Por defecto, PHP incluye soporte de sesiones incorporado. Sin embargo, antes de utilizar sesiones en tu sitio web, debes configurar o comprobar correctamente PHP utilizando las siguientes directivas en el fichero `php.ini` según corresponda:

Directiva	Significado
<code>session.use_cookies</code>	Indica si se deben usar cookies (1) o propagación en la URL (0) para almacenar el SID.
<code>session.use_only_cookies</code>	Se debe activar (1) cuando utilizas cookies para almacenar los SID, y además no quieres que se reconozcan los SID que se puedan pasar como parte de la URL (este método se puede usar para usurpar el identificador de otro usuario).
<code>session.save_handler</code>	Se utiliza para indicar a PHP cómo debe almacenar los datos de la sesión del usuario. Existen cuatro opciones: en ficheros (files), en memoria (mm), en una base de datos SQLite (sqlite) o utilizando para ello funciones que debe definir el programador (user). El valor por defecto (files) funcionará sin problemas en la mayoría de los casos.
<code>session.name</code>	Determina el nombre de la cookie que se utilizará para guardar el SID. Su valor por defecto es PHPSESSID.
<code>session.auto_start</code>	Su valor por defecto es 0, y en este caso deberás usar la función <code>session_start()</code> . para gestionar el inicio de las sesiones. Si usas sesiones en el sitio web, puede ser buena idea cambiar su valor a 1 para que PHP active de forma automática el manejo de sesiones.
<code>session.cookie_lifetime</code>	Si utilizas la URL para propagar el SID, éste se perderá cuando cierres tu navegador. Sin embargo, si utilizas cookies, el SID se mantendrá mientras no se destruya la cookie. En su valor por defecto (0), las cookies se destruyen cuando se cierra el navegador. Si quieres que se mantenga el SID durante más tiempo, debes indicar en esta directiva ese tiempo en segundos.

Directiva	Significado
<code>session.gc_maxlifetime</code>	Indica el tiempo en segundos que se debe mantener activa la sesión, aunque no haya ninguna actividad por parte del usuario. Su valor por defecto es 1440. Es decir, pasados 24 minutos desde la última actividad por parte del usuario, se cierra su sesión automáticamente.

La función **phpinfo()** te ofrece información sobre la configuración actual de las directivas de sesión.

Para saber más ...

En la [documentación de PHP](#) tienes información sobre éstas y otras directivas que permiten configurar el manejo de sesiones.

2.4. Inicio y fin de una sesión

El inicio de una sesión puede tener lugar de dos formas:

- Si has activado la directiva **session.auto_start** en la configuración de PHP, la sesión comenzará automáticamente en cuanto un usuario se conecte a tu sitio web. En caso de que ese usuario ya haya abierto una sesión con anterioridad, y esta no se haya eliminado, en lugar de abrir una nueva sesión se reanudará la anterior. Para ello se utilizará el SID anterior, que estará almacenado en una cookie (recuerda que si usas propagación del SID, no podrás restaurar sesiones anteriores; el SID figura en la URL y se pierde cuando cierras el navegador).
- Si por el contrario, decides no utilizar el inicio automático de sesiones, deberás ejecutar la función **session_start()**. para indicar a PHP que inicie una nueva sesión o reanude la anterior. Aunque anteriormente esta función devolvía siempre true, a partir de la versión 5.3.0 de PHP su comportamiento es más coherente: devuelve false en caso de no poder iniciar o restaurar la sesión.

Como el inicio de sesión requiere utilizar cookies, y éstas se transmiten en los encabezados HTTP, debes tener en cuenta que para poder iniciar una sesión utilizando **session_start()**., tendrás que hacer las llamadas a esta función antes de que la página web muestre información en el navegador.

Además, **todas las páginas** que necesiten utilizar la información almacenada en la sesión, deberán ejecutar la función **session_start()**.

Mientras la sesión permanece abierta, puedes utilizar la variable superglobal **\$_SESSION** para añadir información a la sesión del usuario, o para acceder a la información almacenada en la sesión. Por ejemplo, para contar el número de veces que el usuario visita la página, puedes hacer:

```
// Iniciamos la sesión o recuperamos la anterior sesión existente
session_start();
```



```
// Comprobamos si la variable ya existe
if (isset($_SESSION['visitas']))
    $_SESSION['visitas']++;
else
    $_SESSION['visitas'] = 0;
```

Si en lugar del número de visitas, quisieras almacenar el instante en que se produce cada una, la variable de sesión visitas deberá ser un array y por tanto tendrás que cambiar el código anterior por:

```
// Iniciamos la sesión o recuperamos la anterior sesión existente
session_start();

// En cada visita añadimos un valor al array "visitas"
$_SESSION['visitas'][] = date('d-m-Y H:i:s');
```

Aunque como ya viste, **puedes configurar PHP para que elimine de forma automática los datos de una sesión** a petición del usuario o pasado cierto tiempo. En ocasiones puede ser necesario cerrarla de forma manual en un momento determinado. Por ejemplo, si utilizas sesiones para recordar la información de autenticación, deberás darle al usuario del sitio web la posibilidad de cerrar la sesión cuando lo crea conveniente.

En PHP tienes dos funciones para eliminar la información almacenada en la sesión:

- **session_unset()**. Elimina las variables almacenadas en la sesión actual, pero no elimina la información de la sesión del dispositivo de almacenamiento usado.
- **session_destroy()**. Elimina completamente la información de la sesión del dispositivo de almacenamiento.

¡Atención!

Aunque se use session_destroy(), la sesión no se destruye hasta que termina la ejecución del script, de forma que las variables de sesión estarán disponibles durante el resto de la ejecución del mismo.

2.5. Identificadores de sesión personalizados

El servidor crea un número único para cada nueva sesión. Si quieres obtener el identificador de sesión, puedes usar la función **session_id()**, como muestra el siguiente fragmento de código:

```
<?php
session_start();
echo session_id();
?>
```

Esto da el identificador de sesión actual. La función **session_id()** es interesante porque también puede recibir un argumento, un identificador de sesión. Si quieres reemplazar el

identificador de sesión generado por el sistema por el tuyo propio, puedes suministrarlo como el primer argumento de la función **session_id()**.

```
<?php
session_id(YOUR_SESSION_ID);
session_start();
?>
```

Es importante hacer notar que la función **session_id()** debe estar situada antes que la llamada a **session_start()**. cuando quieras iniciar una sesión con un identificador de sesión personalizado.

Actividad 4.4

Crea una página similar a la de la Actividad 4.1, almacenando en la sesión de usuario los instantes de todas sus últimas visitas. Si es su primera visita, muestra un mensaje de bienvenida. En caso contrario, muestra una lista de la fecha y hora de todas sus visitas anteriores. Añade un botón a la página que permita borrar el registro de visitas.

3. Autenticación de usuarios y control de acceso

Muchas veces **es importante verificar la identidad de los dos extremos de una comunicación**. En el caso de una comunicación web, existen métodos para identificar tanto al servidor en el que se aloja el sitio web, como al usuario del navegador que se encuentra en el otro extremo.

Los sitios web que necesitan emplear identificación del servidor, como las tiendas o los bancos, utilizan el **protocolo HTTPS**. Este protocolo requiere de un **certificado válido**, firmado por una autoridad confiable, que es verificado por el navegador cuando se accede al sitio web. Además, HTTPS utiliza métodos de cifrado para crear un canal seguro entre el navegador y el servidor, de tal forma que no se pueda interceptar la información que se transmite por el mismo.

Para identificar a los usuarios que visitan un sitio web, se pueden utilizar distintos métodos como el **DNI electrónico** o **certificados digitales de usuario**, pero el más extendido es solicitar al usuario cierta información que solo él conoce: la **combinación de un nombre de usuario y una contraseña**.

En la unidad anterior aprendiste a utilizar aplicaciones web para gestionar información almacenada en bases de datos. En la mayoría de los casos es importante implantar en este tipo de aplicaciones web, las que acceden a bases de datos, algún **mecanismo de control de acceso** que obligue al usuario a identificarse. Una vez identificado, se puede limitar el uso que puede hacer de la información.

Así, puede haber sitios web en los que **los usuarios autenticados pueden utilizar sólo una parte de la información** (como los bancos, que permiten a sus clientes acceder únicamente a la información relativa a sus cuentas). Otros sitios web necesitan separar en grupos a los usuarios autenticados, de tal forma que la información a la que accede un usuario depende del grupo en que éste se encuentre. Por ejemplo, una aplicación de gestión de una empresa puede tener un grupo de usuarios a los que permite visualizar la información, y otro grupo de usuarios que, además de visualizar la información, también la pueden modificar.

¡¡ Atención !!

Debes distinguir la autenticación de los usuarios y el control de acceso, de la utilización de mecanismos para asegurar las comunicaciones entre el usuario del navegador y el servidor web. Aunque ambos aspectos suelen ir unidos, son independientes.

En los ejemplos de esta unidad, la información de autenticación (nombre y contraseña de los usuarios) se envía en texto plano desde el navegador hasta el servidor web. Esta práctica es altamente insegura y nunca debe usarse sin un protocolo como HTTPS que permita cifrar las comunicaciones con el servidor web. Sin embargo, la configuración de servidores web que permitan usar el protocolo HTTPS para cifrar la información que reciben y transmiten no

forma parte de los contenidos de este módulo DWES. Por este motivo, durante esta unidad utilizaremos únicamente el protocolo no seguro HTTP.

3.1. Mecanismos de autenticación

3.1.1. Protocolo HTTP

En PHP puedes lanzar cabeceras con la función `header()` y una opción es la autenticación HTTP añadiendo las siguientes líneas en tus páginas:

```
if (!isset($_SERVER['PHP_AUTH_USER'])) {  
    header('WWW-Authenticate: Basic Realm="Contenido restringido"');  
    header('HTTP/1.0 401 Unauthorized');  
    echo "Usuario no reconocido!";  
    exit;  
}
```

Si se pulsa el botón Cancelar, se muestra el mensaje de error que se indica.

Para saber más ...

La [función header\(\)](#) envía encabezados HTTP, pero debe utilizarse antes de cualquier carácter que se envíe al cliente web. En caso contrario, obtendrás un error.

El siguiente ejemplo ilustra este hecho. **Pruébalo.**

```
<html>  
<?php  
/* Esto producirá un error. Fíjese en el html  
 * que se muestra antes que la llamada a header() */  
    header('Location: http://www.example.com/');  
    exit;  
?>
```

Desde PHP puedes acceder a la información de autenticación HTTP que ha introducido el usuario utilizando el array superglobal `$_SERVER` donde entre otras dispones de las siguientes variables:

Valor	Contenido
<code>\$_SERVER['PHP_AUTH_USER']</code>	Nombre de usuario que se ha introducido.
<code>\$_SERVER['PHP_AUTH_PW']</code>	Contraseña introducida

Después del proceso de autenticación, si creas una página web que muestre los valores de estas variables obtendrás algo como lo siguiente:

```
<!DOCTYPE html>  
<!-- Desarrollo Web en Entorno Servidor -->  
<!-- UT 4 : Desarrollo de aplicaciones web con PHP -->  
<!-- Ejemplo: Autenticación HTTP -->  
<html>  
    <head>  
        <meta http-equiv="content-type" content="text/html; charset=UTF-8">
```

```

        <title>Ejemplo UT 4: Autenticación HTTP</title>
    </head>
    <body>
    <?php
        if (isset($_SERVER['PHP_AUTH_USER']) and
isset($_SERVER['PHP_AUTH_PW'])) {
            echo "Nombre de usuario: " . $_SERVER['PHP_AUTH_USER'] . "<br />";
            echo "Contraseña: " . $_SERVER['PHP_AUTH_PW'] . "<br />";
        } else
            echo "No se ha autenticado";
    ?>
    </body>
</html>

```

Si se han introducido las credenciales de acceso se muestra su contenido. En este caso, habría que añadir algún código para comprobar que el nombre de usuario y contraseña son válidos, tal y como veremos a continuación.

3.1.2. Formulario web

Otra opción muy utilizada para permitir la autenticación independiente del protocolo HTTP es **utilizar un formulario y variables de sesión**. Para que el usuario se pueda autenticar, normalmente se suele crear una página con un formulario de login:

```

    <form action="login.php" method="post">
        <label for="uname"><b>Username</b></label>
        <input type="text" placeholder="Enter Username" name="uname"
required>

        <label for="psw"><b>Password</b></label>
        <input type="password" placeholder="Enter Password" name="psw"
required>

        <button type="submit">Login</button>
    </form>

```

En el procesamiento del usuario, si todo está correcto, se crea una variable de sesión que recoja datos de dicho usuario (por ejemplo el nombre de usuario):

```
$_SESSION['usuario'] = $_POST['uname'];
```

Si está sin definir, vacía o nula, se considera que el usuario no está "logueado".

```

if (!empty($_SESSION['usuario'])) {
    echo "Nombre de usuario: " . $_SESSION['usuario'] . "<br />";
} else {
    echo "No se ha autenticado";
}

```

Para **crear zonas** a las que sólo puedan acceder usuarios registrados únicamente habría que comprobar la variable de sesión. Podemos devolver un error ...

```

if (empty($_SESSION['usuario'])) {
    header('HTTP/1.0 401 Unauthorized');
}

```

```
        echo "Usuario no reconocido!";  
        exit;  
    }
```

...o redirigir al usuario a la página de login.

```
    if (empty($_SESSION['usuario'])) {  
        header("Location:login.php");  
        exit;  
    }
```

Para desautenticar a un usuario únicamente habremos de borrar la variable de sesión o asignarle un valor nulo:

```
    unset($_SESSION['usuario']);  
  
    $_SESSION['usuario'] = null;
```

Aunque lo más recomendable es destruir la sesión completa:

```
    session_destroy();
```

3.2 Incorporación de métodos de autenticación a una aplicación web

Si utilizas la función `header()` para forzar al navegador a solicitar credenciales HTTP, el usuario introducirá un nombre y una contraseña. Pero el servidor no verificará esta información; deberá ser el desarrollador web quien provea un método para comprobar que las credenciales que ha introducido el usuario son correctas.

El método más simple es incluir en el código PHP de tu página las sentencias necesarias para **comparar los datos introducidos con otros datos fijos**. Por ejemplo, para permitir el acceso a un usuario **dwes** con contraseña **abc123**, puedes hacer:

```
    if (($_SERVER['PHP_AUTH_USER'] != 'dwes' || $_SERVER['PHP_AUTH_PW'] !=  
    'abc123')) {  
        header('WWW-Authenticate: Basic Realm="Contenido restringido");  
        header('HTTP/1.0 401 Unauthorized');  
        echo "Usuario no reconocido!";  
        exit;  
    }
```

Recuerda que el código PHP no se envía al navegador, por lo que la información de autenticación que introduzcas en el código no será visible por el usuario. Sin embargo, esto hará tu código menos portable. Si necesitas modificar el nombre de usuario o la contraseña, tendrás que hacer modificaciones en el código. Además, no podrás permitir al usuario introducir su propia contraseña.

Una solución mejor es utilizar un **almacenamiento externo para los nombres de usuario y sus contraseñas**. Para esto podrías emplear un fichero de texto, o mejor aún, una base de datos. La información de autenticación podrá estar aislada en su propia base de datos, o compartir espacio de almacenamiento con los datos que utilice tu aplicación web.

Aunque se podrían almacenar las contraseñas en texto plano, es **mejor hacerlo almacenando su hash**. De esta manera si alguien robara las bases de datos no obtendría las contraseñas. **Para comprobar si una contraseña es correcta, calculamos su hash y lo comparamos con el que se encuentra almacenado en la base de datos.**

Para saber más ...

Con frecuencia no se calcula el hash de la clave directamente, sino que antes se le concatena una cadena de caracteres conocida como *[sal]* ([https://es.wikipedia.org/wiki/Sal_\(criptografia\)](https://es.wikipedia.org/wiki/Sal_(criptografia)))

En PHP puedes usar la **función md5()** para calcular el hash MD5 de una cadena de texto. No obstante, aunque suele usarse mucho, **no es muy recomendable** usarla para el encriptado de contraseñas. Se recomienda usar el **API de contraseñas**, el cual además genera la sal de manera aleatoria.

```
<?php
$usuario = "lola25";
$nombre = "Lola Pérez";
$clave = password_hash("12341234", PASSWORD_DEFAULT);
$bd->exec('INSERT INTO usuarios (user, password, name)
          VALUES ("' . $usuario . "', "' . $clave . "', "' . $nombre .
          "')');

$sql = "SELECT user, password FROM usuarios WHERE user = :username";
$consulta = $bd->prepare($sql);
$consulta->execute(["username" => $_POST['usuario']]);
$usuario = $consulta->fetch();
if (!empty($usuario) and password_verify($_POST['clave'],
$usuario['password'])) {
    $_SESSION['usuario'] = $usuario['user'];
    echo '<h1>Bienvenido</h1>';
} else {
    echo 'Usuario o clave incorrectos';
}
```

Actividad 4.5

Crea un formulario de inicio de sesión. Si la autenticación es correcta, la página debe mostrar un mensaje de bienvenida. En caso contrario, debe mostrar el formulario de inicio de sesión y un mensaje de error.

Los usuarios se deben almacenar en la base de datos, teniendo cuidado de no almacenar la contraseña en texto plano sino su hash. Se propone usar el siguiente código SQL:

```
SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
SET time_zone = "+00:00";

CREATE TABLE `usuarios` (
  `id_usuario` int(10) UNSIGNED NOT NULL,
  `user` varchar(255) COLLATE utf8_bin NOT NULL UNIQUE,
  `password` varchar(255) COLLATE utf8_bin NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
```

```
ALTER TABLE `usuarios` ADD PRIMARY KEY (`id_usuario`);

ALTER TABLE `usuarios`
  MODIFY `id_usuario` int(10) UNSIGNED NOT NULL AUTO_INCREMENT,
  AUTO_INCREMENT=1;
```